

RAP Cloud 課程 6：Composition / Association (Header-Item)

Lecture

這一課要證明的事情

舊 On-Premise 課程 rap08 教了 Composition / Association (Header-Item 兩層結構)，但因為系統版本落後，踩到兩個跟官方教材不一樣的地方：① 子實體的 `lock dependent by _<別名>` 官方語法在那個系統編譯失敗，被迫改用 `lock dependent ( 欄位 = 欄位 )`；② Managed CUD 本來就執行不了 (白名單限制)，只能用 Unmanaged，而 Unmanaged 版本框架完全不處理 **Cascading Delete**，要自己手寫「先刪子表、再刪父表」的邏輯。

這一課用 Managed BDEF (延續 rc02 已驗證「Managed CUD 真的能跑」) 重新做一次同樣的 Header-Item 結構，兩個問題都要重新驗證：這個 Cloud 環境的官方現行語法能不能直接用、Managed Composition 的 Cascading Delete 是不是也跟 rc05 的 Draft 一樣完全由框架處理。

結論先講：兩個問題的答案都是好消息——`lock dependent by _Header` 一次就編譯成功，不需要 On-Premise 那個 workaround；`ZBP_I_RC06_ORDER` 全程維持空殼，Managed Composition 的 Cascading Delete 完全由框架自動處理，不用寫任何 ABP 邏輯。但過程中也踩到一個跟舊課程結論相反的坑：EML 呼叫端連結父子關聯要用 `%cid_ref`，不是舊課程教的 `%key`——這次是舊課程的結論被推翻，值得完整記錄。

CDS View Entity 語法：比舊課程簡潔很多

查證官方文件 (`ABENEML_MODIFY_CUSTOM_OP_ABEXA` / `ABENDERIVED_TYPES_CID_CIDREF_ABEXA`) 確認 View Entity 語法：

Header (Root · composition)：

```
define root view entity ZI_RC06_ORDER as select from zrc06_order
composition [0..*] of ZI_RC06_ORDER_I as _Item
{
  key order_id,
  description,
  created_at,
  created_by,

  _Item
}
```

Item (Dependent Child · association to parent)：

```
define view entity ZI_RC06_ORDER_I as select from zrc06_order_i
association to parent ZI_RC06_ORDER as _Header
  on $projection.order_id = _Header.order_id
```

```

{
  key order_id,
  key item_id,
  material_desc,
  quantity,

  _Header
}

```

跟舊課程 rap08 用的舊式 `define view / define root view` (不帶 `entity`) 相比，完全不需要 `@AbapCatalog.preserveKey: true / @ObjectModel.compositionRoot: true` 這兩個舊式 annotation——這兩個 annotation 是 V1 (`define root view`) 語法用來人工標註「這是 Composition Root」的手段，V2 (`define root view entity`) 語法裡 `root` 這個關鍵字本身已經表達了這個語意，不需要額外標註。

Eclipse 精靈的兩個現成模板剛好對應這兩種角色：建 Header CDS View 時選 `defineRootViewEntity`；建 Item CDS View 時選 `defineViewEntityWithToParentAssociation`——這個模板會直接生成 `association to parent ... on $projection...` 的骨架，比舊課程手動照抄範例更省事。

⚠ 互相引用的 **Mass Activation**，MCP 工具做不到，一定要在 **VS Code** 手動處理：Header 的 `composition` 引用 Item、Item 的 `association to parent` 引用 Header，兩個 CDS View 互相依賴，任何一個單獨啟用都會報「data source does not exist or is not active」。查證 `abap-remote-fs` 擴充套件官方文件，明確記載這個限制：「Mass Activation：You must select objects from a dialog; activation is not automatic」——這代表 Claude 用 MCP 工具無法觸發互相依賴物件的批次啟用，這一步一定要請使用者在 **VS Code** 編輯器裡手動按啟用，讓「**Mass Activation**」對話框跳出來，勾選兩個一起確認，才能解開循環依賴。這是這門課第一次遇到「MCP 工具連 CDS View 都做不到」的情境（先前的限制都只發生在 Class 的 Local Types）。

Managed BDEF：兩個 `define behavior for` 區塊

```

managed implementation in class zbp_i_rc06_order unique;
strict ( 2 );

define behavior for ZI_RC06_ORDER alias Header
persistent table zrc06_order
lock master
authorization master ( none )
{
  create;
  update;
  delete;

  association _Item { create; }

  field ( readonly : update ) order_id;
  field ( readonly )         created_at, created_by;
  field ( mandatory )        description;
}

```

```

define behavior for ZI_RC06_ORDER_I alias Item
persistent table zrc06_order_i
lock dependent by _Header
authorization dependent by _Header
{
  update;
  delete;

  association _Header;

  field ( readonly ) order_id;
  field ( mandatory ) material_desc, quantity;
}

```

- **lock dependent by _Header**——這個 **Cloud** 環境一次就編譯成功，不需要舊課程 rap08 的 workaround (**lock dependent (order_id = order_id)**)。這是繼 rc03 (Determination / Validation 用官方現行語法) 之後，這門課第二次直接驗證「這個環境版本領先，官方現行語法直接可用」。
- **authorization dependent by _Header**——子實體的權限宣告要對應到父實體怎麼處理：既然 Header 是 **authorization master(none)** (完全不作權限檢查)，Item 就繼承這個決定，用 **authorization dependent by _Header** 表明「權限判斷跟著父實體走」。strict(2) 要求每個實體都要明確宣告 authorization 角色，子實體用 **dependent by <關聯別名>**，跟父實體的 **master(...)** 是不同的宣告方式。
- **association _Item { create; }**——宣告父實體允許透過這個關聯建立子實體 (Create-by-Association · CBA)。
- ⚠ **item_id (Item 自己的主鍵) 不能標 field(readonly)**——只有 **order_id** (從父實體帶過來的外鍵) 才該是 readonly。啟用時 Eclipse 會給出明確警告：The key field "ITEM_ID" should be flagged as "readonly" or "readonly:update"。——這句警告字面上建議你標記它，但照做反而會出錯：CBA 建立子實體時，**item_id** 是呼叫端透過 EML 傳進來的資料，不是框架自動決定的值，標成 readonly 會直接擋下這次建立 (這是延續舊課程 rap08 已經驗證過的規則：使用者需要輸入的欄位不能是 readonly，只有框架/邏輯決定值的欄位才能標)。這是一個「警告文字本身會誤導你走錯方向」的案例，判斷 **readonly** 該不該標，要看欄位語意 (誰決定這個值)，不要照單全收編輯器的建議。

EML Create-by-Association : %cid_ref，不是 %key——推翻舊課程 rap08 的結論

舊課程 rap08 在 On-Premise 系統上實測發現：官方範例教的 %cid_ref 寫法完全不生效 (語法編譯正常、EML 呼叫沒有任何錯誤，但子實體就是建不出來)，改用 %key-order_id 才成功，因此下了「父實體 Key 是使用者自訂值時用 %key，只有框架自動編號才需要 %cid_ref」的結論。

這一課完全重現了同樣的排錯過程，但結論相反：先用 %key-order_id = lv_order_id 照抄舊課程的寫法，ABAP Unit 測試兩個都在第一個斷言就失敗——FAILED 表格裡有一筆記錄，但 REPORTED 完全是空的，沒有任何錯誤訊息可看，跟舊課程當年遇到的「靜默失敗」症狀一模一樣，只是這次是反過來的版本。查證官方文件 ABENDERIVED_TYPES_CID_CIDREF_ABEXA (現行 Cloud 文件) 的完整範例才發現：即使父實體的 Key 是呼叫端明確指定的值 (key_field = 1，完全不是自動編號)，官方範例照樣用 %cid_ref = 'cid1' 反查父實體，不是 %key。改用 %cid_ref 之後，兩個測試一次就通過。

```

" ---- 一次 EML 呼叫同時建立 Header 跟兩筆 Item ----
MODIFY ENTITIES OF zi_rc06_order
  ENTITY Header
    CREATE FIELDS ( order_id description )
    WITH VALUE #( ( %cid = 'H1' order_id = lv_order_id description = 'Demo Order' ) )

  ENTITY Header
    CREATE BY \_Item
    FIELDS ( item_id material_desc quantity )
    WITH VALUE #( ( %cid_ref = 'H1'
                    %target = VALUE #(
                        ( %cid = 'I1' item_id = '0010' material_desc = 'Widget A'
                        quantity = '5' )
                        ( %cid = 'I2' item_id = '0020' material_desc = 'Widget B'
                        quantity = '3' ) ) ) ) )

  FAILED DATA(ls_failed)
  REPORTED DATA(ls_reported).

COMMIT ENTITIES RESPONSE OF zi_rc06_order ...

```

- **%cid_ref = 'H1'** 對應第一個 **ENTITY Header CREATE** 那一系列的 **%cid = 'H1'**——用同一個 EML 陳述式裡前面已經指定過的暫時代碼，反查「這批 Item 要掛在哪一個剛建立的 Header 底下」。
- 一句話更正：**%cid_ref** 的用途是「反查同一個 EML 陳述式裡稍早建立的實例」，不管父實體的 **Key** 是不是使用者自訂——這是這個判斷準則的完整版本，舊課程 **rap08** 記錄的「只有框架自動編號才需要 **%cid_ref**」是不完整、甚至方向相反的結論，很可能是那個系統版本或 Unmanaged 實作路徑上的特有差異（Unmanaged 的 Handler 是自己手寫 **%key** / **%target** 存取邏輯，跟 Managed 框架內建處理的機制本來就不是同一條路徑，這點沒有進一步查證，誠實記錄矛盾而不是硬湊一個解釋）。這一課的教訓是：同一個坑，換一個環境 / 換一種實作方式（**Managed vs Unmanaged**），答案可能完全相反，不能把「上一次踩坑學到的結論」直接套用到新環境，每次都要重新驗證。

Cascading Delete：Managed Composition 自動處理，不用寫任何程式碼

```

" ---- DELETE Header ----
MODIFY ENTITIES OF zi_rc06_order
  ENTITY Header
  DELETE
    FROM VALUE #( ( %key-order_id = lv_order_id ) )
  FAILED DATA(ls_failed_delete)
  REPORTED DATA(ls_reported_delete).

COMMIT ENTITIES RESPONSE OF zi_rc06_order ...

```

舊課程 rap08 (Unmanaged) 的 **delete** Handler Method 要自己寫兩行 **DELETE FROM** (先刪 Item 子表、再刪 Header 父表)，並且在講義裡特別強調「Unmanaged 完全沒有框架自動處理，**association _Item {**

`create; }` 只宣告了『允許建立』，跟刪除完全無關，不自己處理就會留下孤兒 Item 資料列」。

這一課驗證：**Managed** 完全不是這樣——`ZBP_I_RC06_ORDER` 全程維持純空殼（`ABSTRACT FINAL FOR BEHAVIOR OF zi_rc06_order`，`IMPLEMENTATION` 區塊完全空白，沒有任何 Local Types 內容），單純呼叫 `EML DELETE` 對 Header，`COMMIT ENTITIES` 之後直接下 SQL 查兩張表：`zrc06_order`（Header）跟 `zrc06_order_i`（Item）都確認清空——**Composition** 的「擁有」語意（父實體刪除，子實體連帶刪除）由 **Managed** 框架自動保證，不需要開發者自己寫任何 **Cascading Delete** 邏輯。這是繼 rc05（Draft 完全框架處理）之後，這門課第二次驗證「Managed Runtime 真的能跑」帶來的具體好處——不只是「CRUD 能執行」，連「維護資料完整性的複雜邏輯」都內建。

ABAP Unit 執行結果

```
Unit Test Results for CLAS/I ZCL_RC06_ORDER_TEST.main
Status: ALL TESTS PASSED
Total: 2 | Passed: 2 | Failed: 0

[PASS] ZCL_RC06_ORDER_TEST
  [PASS] CREATE_BY_ASSOCIATION (0.230s)
  [PASS] DELETE_CASCADES_TO_ITEMS (0.120s)
```

`CREATE_BY_ASSOCIATION`：一次 EML 呼叫同時建立 Header + 兩筆 Item，`COMMIT` 成功後分別用 `READ ENTITIES` 讀回 Header / Item 驗證資料正確。`DELETE_CASCADES_TO_ITEMS`：先用 CBA 建立 Header + 1 筆 Item（直接下 SQL 確認 Item 真的存在），再 `DELETE` Header，`COMMIT` 之後確認 Header / Item 都從資料庫消失——驗證 Cascading Delete 真的是框架自動處理的，不是巧合。

學習目標

- 能寫出 Composition / Association 的 View Entity 語法：`composition [0..*] of <子View> as _<別名> (父, Root)`、`association to parent <父View> as _<別名> on $projection.<欄位> = _<別名>.<欄位> (子, Dependent)`，知道 V2 語法不需要 `@AbapCatalog.preserveKey / @ObjectModel.compositionRoot` 這兩個 V1 專屬 annotation
- 知道 Eclipse 精靈有專屬模板：Header 用 `defineRootViewEntity`、Item 用 `defineViewEntityWithToParentAssociation`
- 能寫出這個環境適用的 BDEF：`lock dependent by _<別名>`（官方現行語法直接可用，對照舊課程需要的 workaround）、`authorization dependent by _<別名>`
- 知道子實體 readonly 欄位的判斷原則：使用者需要輸入的欄位（如子實體自己的 Key）不能標 readonly，即使 Eclipse 警告建議你標
- 能講出這一課最重要的更正：EML Create-by-Association 連結父子關聯要用 `%cid_ref`，不是舊課程 rap08 的 `%key`——即使父實體 Key 是使用者自訂值，官方現行文件範例照樣用 `%cid_ref`，知道「上一個環境學到的結論不能直接套用到新環境」這個方法論教訓
- 能講出 Managed Composition 的 Cascading Delete 完全由框架處理，不需要任何 ABP 實作，對照舊課程 Unmanaged 版本要自己手寫先刪子表、再刪父表的邏輯
- 知道兩個 CDS View 互相依賴時的 Mass Activation，MCP 工具做不到（官方文件明講的限制），一定要在 VS Code 手動觸發批次啟用對話框

物件清單

物件	名稱	型別
Header 正式表	ZRC06_ORDER	TABL/DT
Item 正式表	ZRC06_ORDER_I	TABL/DT
Header CDS View (Composition Root)	ZI_RC06_ORDER	DDLS/DF
Item CDS View (Dependent)	ZI_RC06_ORDER_I	DDLS/DF
Managed Behavior Definition (含兩個 behavior 區塊)	ZI_RC06_ORDER	BDEF/BDO
Implementation Class (純空殼，無 Local Types 內容)	ZBP_I_RC06_ORDER	CLAS/OC
ABAP Unit 測試類別	ZCL_RC06_ORDER_TEST	CLAS/OC

套件：ZRAPCLOUD。全部物件由使用者在 Eclipse / VS Code 建立空殼，Claude 用 MCP 寫入完整內容並驗證；兩個 CDS View 互相依賴的批次啟用由使用者在 VS Code 手動完成。

驗證方式

1. `get_abap_diagnostics` 確認全部物件無語法錯誤
2. `abap_activate` 全部回報 `Activation successful` (CDS View 的循環依賴由使用者在 VS Code 用 Mass Activation 對話框一次解開)
3. `run_unit_tests` 對 `ZCL_RC06_ORDER_TEST` 執行，`ALL TESTS PASSED` (2/2，見上方完整輸出)
4. `delete_cascades_to_items` 額外直接下 SQL 查 `zrc06_order` / `zrc06_order_i` 兩張表，獨立於 RAP 框架之外驗證 Cascading Delete 真的發生

思考題

1. 這一課的 `%cid_ref` 發現推翻了舊課程 rap08 的結論。如果你手上同時有這個 Cloud 環境 (Managed) 跟舊 On-Premise 系統 (Unmanaged)，可以怎麼設計一個實驗，分別驗證「是 Managed vs Unmanaged 造成的差異」還是「單純是這兩個系統版本落差造成的差異」？ (提示：這一課沒有做這個交叉驗證，只誠實記錄了矛盾)
2. `association_Item { create; }` 只宣告了 CBA (建立)。如果想讓 Item 也能透過 Header 的關聯做 Read-by-Association (例如 `READ ENTITIES ... ENTITY Header BY \_Item ...`)，BDEF 要多宣告什麼？ (提示：查官方文件 `ABENBDL_ASSOC_STAND_OPS`，這一課的驗證方式是直接對 `ENTITY Item` 做 `READ ENTITIES`，不是透過關聯導覽)
3. 這一課驗證了 Cascading Delete，但沒有驗證 Cascading Update / 其他情境 (例如更新 Header 時是否需要連動處理 Item 的某些欄位)。如果你要自己延伸這個 Header-Item 結構，加一個「Header 的 `description` 改變時，自動在所有 Item 的某個欄位打上時間戳」的需求，你會用 rc03 教過的哪個機制實作？ (提示：這是一個典型的 `determination ... on save { field ... }` 應用場景，但要注意這次是「父實體欄位變更、影響子實體」，不是同一個實體內部)

答案

見 `zrc06_order.tabl.abap` (Header 表)、`zrc06_order_i.tabl.abap` (Item 表)、
`zi_rc06_order.ddls.abap` (Header CDS View)、`zi_rc06_order_i.ddls.abap` (Item CDS View)、
`zi_rc06_order.bdef.abap` (含兩個 behavior 區塊)、`zbp_i_rc06_order.clas.abap` (純空殼)、

zcl_rc06_order_test.clas.abap (create_by_association / delete_cascades_to_items 兩個測試方法) 。SAP 端物件套件 ZRAPCLOUD · run_unit_tests 執行結果：ALL TESTS PASSED (2/2) 。